ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN 1
MÔN NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

TOPIC 1: MINESWEEPER

Ngành: KHOA HỌC MÁY TÍNH

GVHD: NGUYỄN QUỐC MINH

—o0o—

Lớp: A01

SV: LÝ QUỐC HÀO (2352294)

SV: NGUYỄN NGỌC PHÁT (2352887)

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

Mục lục

Danh sách sinh viên thực hiện **1**

1 Tổng quan bài tập lớn **2**
1.1 Mô tả bài toán Minesweeper . . . 2
1.2 Mô hình trạng thái bàn cờ . . . 2
1.2.1 Cấu trúc dữ liệu và biểu diễn tọa độ . . . 2
1.2.2 Các biến trạng thái quan sát và tương tác (*Observable State*) . . . 3
1.2.3 Vòng đời và điều kiện chuyển trạng thái thắng/thua (*State Transitions & Victory/Defeat*) . 3
1.3 Cơ chế bảo vệ lượt đầu tiên (First-move protection) . . . 3

2 THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG **4**
2.1 Tổng quan giao diện . . . 4
2.1.1 Công nghệ nền tảng và phong cách thiết kế . . . 4
2.1.2 Cơ chế hiển thị bảng cờ động bằng `tk.Canvas` . . . 4
2.1.3 Bảng điều khiển và chỉ số thống kê dạng thẻ (Information Dashboard) . . . 4
2.2 Chức năng điều khiển và thao tác . . . 4
2.2.1 Chức năng tạo ván mới . . . 4
2.2.2 Thao tác tương tác . . . 5
2.2.3 Tính năng hỗ trợ (Reasoning Assistant) . . . 5
2.3 Phân tách dữ liệu . . . 5

3 THUẬT TOÁN MỞ RỘNG VÙNG AN TOÀN **5**
3.1 Lựa chọn thuật toán: Breadth-First Search (BFS) vs Depth-First Search (DFS) . . . 5
3.1.1 Bảng so sánh đặc tính kỹ thuật: BFS và DFS trong Flood-fill . . . 5
3.1.2 Lý do lựa chọn BFS . . . 6
3.2 Mã giả cho BFS . . . 6
3.3 Mô phỏng thuật toán BFS . . . 7
3.4 Đánh giá độ phức tạp thuật toán . . . 8

4 PHÂN TÍCH HÀM HEURISTIC VÀ SUY LUẬN LOGIC **8**
4.1 Cơ chế suy diễn logic (Deduction Rules) . . . 8
4.2 Hàm Heuristic ước lượng rủi ro cục bộ (Estimated Local Risk) . . . 8
4.3 Đánh giá tính hợp lệ của Heuristic . . . 9

5 Kết quả thực nghiệm và kiểm thử hệ thống **9**
5.1 Phương pháp kiểm thử tự động (Automated Checks) . . . 9
5.1.1 Cách thực thi kiểm thử . . . 9
5.2 Hình ảnh minh họa giao diện thực tế . . . 9
5.3 Đánh giá độ ổn định và hướng dẫn thực thi mã nguồn . . . 12
5.3.1 Hướng dẫn chạy mã nguồn . . . 12

6 TÀI LIỆU THAM KHẢO **12**

Danh sách sinh viên thực hiện

STT	Họ và tên	MSSV	Mức độ hoàn thành
1	Lý Quốc Hào	2352294	100%
2	Nguyễn Ngọc Phát	2352887	100%

1 Tổng quan bài tập lớn

1.1 Mô tả bài toán Minesweeper

Minesweeper là một trò chơi giải đố logic kinh điển được xây dựng trên một lưới ô chữ nhật kích thước $R \times C$ (R hàng và C cột). Trên bảng có tổng cộng M quả mìn được bố trí ngẫu nhiên tại các vị trí phân biệt, với $1 \leq M < R \times C$. Toàn bộ vị trí mìn ban đầu đều được giữ bí mật; do đó, đây là một môi trường quan sát được một phần (partially observable).

Mục tiêu của người chơi là lật mở toàn bộ $R \times C - M$ ô an toàn mà không kích nổ bất kỳ quả mìn nào.

Các quy tắc tương tác và cơ chế phản hồi:

- **Mở ô an toàn (Reveal):** Khi người chơi mở một ô an toàn, trò chơi hiển thị một số nguyên từ 0 đến 8, cho biết số lượng mìn trong tối đa 8 ô lân cận (lân cận Moore: ngang, dọc và 4 đường chéo). Con số này đóng vai trò là dữ liệu ràng buộc , giúp người chơi suy luận vị trí các quả mìn chưa biết.
- **Mở rộng vùng an toàn (Flood-fill):** Nếu ô được mở không giáp quả mìn nào , hệ thống tự động kích hoạt thuật toán tìm kiếm theo chiều rộng (BFS) để lật mở toàn bộ vùng các ô 0 liên thông cùng đường biên gồm những ô có số lân cận mìn.
- **Cắm cờ (Flagging):** Người chơi có thể đánh dấu cờ tại các ô nghi vấn có mìn. Lá cờ đóng vai trò là ghi chú của người chơi , hỗ trợ ghi nhớ và tránh vô tình mở nhầm ô; thao tác này không làm thay đổi vị trí thực tế của mìn.

Điều kiện kết thúc:

- **Thất bại (Defeat):** Người chơi mở trúng ô chứa mìn; mìn phát nổ và ván đấu kết thúc ngay lập tức.
- **Chiến thắng (Victory):** Người chơi mở thành công tất cả các ô an toàn.

Minesweeper không đơn thuần là một trò chơi may rủi mà còn là một bài toán suy luận thỏa mãn ràng buộc logic (Constraint Satisfaction Problem - CSP), kết hợp thuật toán duyệt đồ thị và ước lượng heuristic khi gặp các tình huống bất định.

1.2 Mô hình trạng thái bàn cờ

Để mô hình hóa trò chơi Minesweeper dưới dạng bài toán tìm kiếm và suy diễn logic trong trí tuệ nhân tạo, hệ thống cần thiết lập một không gian trạng thái (*state space*) chặt chẽ. Mô hình trạng thái bàn cờ được đóng gói bên trong lớp `Board` (mô-đun `engine.py`), phân định rõ ràng giữa thông tin môi trường thực tế (*ground truth*) và thông tin người chơi/tác tử quan sát được (*agent observation*).

1.2.1 Cấu trúc dữ liệu và biểu diễn tọa độ

- **Tọa độ ô cờ (`Cell`):** Mỗi ô trên bàn cờ được định danh duy nhất bằng một tuple chỉ số nguyên `(row, col)` theo hệ tọa độ 0:
$$\text{Cell} = (r, c) \quad \text{với } 0 \leq r < R \text{ và } 0 \leq c < C.$$
(Trên giao diện người dùng, tọa độ được chuyển đổi sang 1-indexed để thuận tiện quan sát.)
- **Tập hợp lân cận (`Neighbors`):** Mỗi ô (r, c) tương tác với tối đa 8 ô lân cận xung quanh (lân cận Moore: ngang, dọc và chéo), được tính thông qua hàm `neighbors(rows, cols, cell)`:
$$\mathcal{N}(r,c) = \{(nr, nc) \mid |nr - r| \leq 1,\ |nc - c| \leq 1,\ (nr, nc) \neq (r, c),\ 0 \leq nr < R,\ 0 \leq nc < C\}.$$
- **Không gian lưu trữ thực tế (*Ground Truth* – trạng thái riêng tư):**
  - `_mines: set[Cell]`: Tập hợp tọa độ các ô thực sự chứa mìn do hệ thống phân bổ ngẫu nhiên. Tập hợp này có kích thước cố định $|\texttt{_mines}| = M$.
  - `_counts: dict[Cell, int]`: Từ điển ánh xạ mỗi tọa độ ô (r, c) với số lượng mìn trong các ô lân cận:
$$\texttt{_counts}[(r,c)] = \sum_{n \in \mathcal{N}(r,c)} \mathbb{I}(n \in \texttt{_mines}),$$
trong đó $\mathbb{I}$ là hàm chỉ thị, nhận giá trị 1 nếu ô có mìn và 0 nếu ô an toàn.

1.2.2 Các biến trạng thái quan sát và tương tác (*Observable State*)

Người chơi và bộ suy luận logic tương tác với môi trường thông qua các biến trạng thái quan sát:

- `revealed: set[Cell]`: Tập hợp các ô an toàn đã được lật mở thành công. Khi một ô được mở, giá trị số mìn lân cận `_counts[cell]` trở thành tri thức công khai (*revealed clue*).
- `flags: set[Cell]`: Tập hợp các ô chưa mở đang được người chơi đánh dấu cờ. Cần nhấn mạnh rằng `flags` chỉ là nhãn ghi chú chủ quan của người chơi; hệ thống **không** coi cờ là bằng chứng mìn tuyệt đối khi thực hiện suy diễn logic.
- `exploded: Cell | None`: Lưu trữ tọa độ quả mìn phát nổ khi người chơi thao tác sai (mặc định là `None`).

1.2.3 Vòng đời và điều kiện chuyển trạng thái thắng/thua (*State Transitions & Victory/Defeat*)

Trạng thái tổng thể của bàn cờ được quản lý thông qua biến `status` với các giá trị `ready`, `playing`, `won` và `lost`, theo máy trạng thái hữu hạn (*Finite State Machine*) như sau:

1. **Khởi tạo (`ready`):** Bàn cờ được tạo mới với các kích thước R, C và M. Chưa có quả mìn nào được rải ngẫu nhiên trên bảng nhằm phục vụ cơ chế bảo vệ lượt đi đầu tiên.
2. **Đang chơi (`playing`):** Trạng thái này được kích hoạt ngay khi người chơi thực hiện thao tác mở ô đầu tiên. Tại thời điểm đó, mìn được tạo và trò chơi chuyển sang trạng thái tương tác chính.
3. **Thua cuộc (`lost` – Defeat):** Xảy ra khi người chơi mở ô `cell` nằm trong tập mìn thực tế:

$$\texttt{cell} \in \texttt{_mines} \implies \texttt{status} \leftarrow \texttt{lost}, \quad \texttt{exploded} \leftarrow \texttt{cell}.$$

   Khi trạng thái chuyển sang `lost`, mọi thao tác mở ô và cắm cờ tiếp theo đều bị vô hiệu hóa (`finished = True`).
4. **Chiến thắng (`won` – Victory):** Xảy ra khi toàn bộ các ô an toàn trên bàn cờ đã được lật mở mà không chạm vào quả mìn nào:

$$|\texttt{revealed}| = \texttt{safe_total} = R \times C - M \implies \texttt{status} \leftarrow \texttt{won}.$$

   Tiêu chí thắng hoàn toàn dựa trên số ô an toàn đã mở (`revealed`), không phụ thuộc vào việc người chơi đã cắm cờ đủ hay đúng vị trí tất cả các quả mìn hay chưa. Điều này đảm bảo tính chính xác và khách quan theo luật thi đấu quốc tế của Minesweeper.

1.3 Cơ chế bảo vệ lượt đầu tiên (First-move protection)

Cơ chế khởi tạo trễ (Lazy mine placement): Khi người chơi nhấp mở ô đầu tiên trong trạng thái `ready`, hàm `_place_mines(first)` mới được kích hoạt để rải mìn.

Vùng bảo vệ an toàn (Safe Ring): Hệ thống bảo vệ ô được nhấp đầu tiên cùng toàn bộ các ô lân cận xung quanh. Tập hợp các ô được bảo vệ được biểu diễn như sau:

$$\texttt{protected} = \{\texttt{first}\} \cup \mathcal{N}(\texttt{first}),$$

trong đó $\mathcal{N}(\texttt{first})$ là tập hợp các ô lân cận của ô đầu tiên. Các quả mìn được lấy mẫu ngẫu nhiên bằng `random.sample` từ tập ứng viên không thuộc `protected`, với điều kiện tập ứng viên còn đủ ô để bố trí toàn bộ số mìn.

Lợi ích: Cơ chế này đảm bảo nước đi đầu tiên mở ra một ô có số đếm bằng 0 (`clue = 0`), từ đó kích hoạt thuật toán BFS để mở rộng một vùng an toàn lớn. Nhờ vậy, người chơi không phải đoán mò hoặc có nguy cơ thua ngay ở lượt đầu tiên.

2 THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

2.1 Tổng quan giao diện

2.1.1 Công nghệ nền tảng và phong cách thiết kế

- **Thư viện chuẩn, tính tương thích cao:** Toàn bộ hệ thống giao diện được lập trình bằng thư viện chuẩn `tkinter` và `ttk` có sẵn trong Python 3.10 trở lên.
- **Ngôn ngữ thiết kế hiện đại (Flat UI):** Giao diện sử dụng theme `clam` của `ttk`, kết hợp bảng màu đồ họa phẳng:
  - Nền chính dịu mắt và thanh tiêu đề màu xanh đen.
  - Màu điểm nhấn thương hiệu được dùng cho các nút hành động chính.
  - Các số đếm mìn lân cận từ 1 đến 8 có màu riêng để dễ nhận diện, chẳng hạn số 1 màu xanh lam đậm, số 2 màu xanh lá cây, số 3 màu đỏ và số 4 màu tím.

2.1.2 Cơ chế hiển thị bảng cờ động bằng `tk.Canvas`

Thay vì sử dụng một lưới các nút bấm `tk.Button` rời rạc, vốn dễ làm giao diện chậm và thiếu linh hoạt khi kích thước bàn cờ lớn, ứng dụng vẽ toàn bộ bảng cờ trên một vùng đồ họa đơn nhất là `tk.Canvas`.

- **Co giãn theo kích thước lưới (`cell_size`):** Để bảng cờ vừa với khung hiển thị cố định 490×540 px mà không vỡ bố cục, kích thước cạnh mỗi ô (`cell_size`) được tính toán động theo số hàng của từng chế độ chơi:
  - Cấp độ 5×5: kích thước ô là 64 px, tạo khoảng cách biên lớn và hiển thị thoáng.
  - Cấp độ 9×9: kích thước ô là 47 px.
  - Cấp độ 16×16: kích thước ô tự động thu nhỏ còn 28 px để hiển thị trọn vẹn 256 ô.
- **Ánh xạ tọa độ pixel sang ô cờ:** Hệ thống dành lề hiển thị tọa độ hàng và cột ở mép trên, mép trái (`margin = 24 px`). Khi có sự kiện nhấp chuột, hàm `_event_cell()` chia nguyên tọa độ chuột (x, y) trên Canvas để xác định chỉ số hàng và cột tương ứng (r, c):

$$c = \left\lfloor \frac{x - \text{margin}}{\text{cell_size}} \right\rfloor, \qquad r = \left\lfloor \frac{y - \text{margin}}{\text{cell_size}} \right\rfloor.$$

2.1.3 Bảng điều khiển và chỉ số thống kê dạng thẻ (Information Dashboard)

Phía trên bàn cờ là khu vực thông số trực quan gồm bốn thẻ trắng (*Dashboard Cards*), được bố trí đều bằng layout `grid()`:

1. **Trạng thái ván đấu (`STATUS`):** Hiển thị tình trạng hiện thời của trò chơi: `READY`, `PLAYING`, `WON` hoặc `LOST`.
2. **Số ô an toàn (`SAFE CELLS`):** Đếm số ô an toàn đã mở trên tổng số ô an toàn cần hoàn thành, theo dạng `X / Y`, giúp người chơi theo dõi tiến độ.
3. **Chỉ số cờ và mìn (`FLAGS / MINES`):** Thống kê số lượng cờ đã cắm so với tổng số mìn của màn chơi, chẳng hạn `2 / 10`.
4. **Thời gian chơi (`TIME`):** Đồng hồ kỹ thuật số định dạng `MM:SS`, tự động đếm giây bằng phương thức `root.after(1000)` kể từ khi người chơi mở ô đầu tiên và dừng lại khi ván cờ kết thúc.

2.2 Chức năng điều khiển và thao tác

2.2.1 Chức năng tạo ván mới

Ứng dụng hỗ trợ ba chế độ chơi mặc định, được lựa chọn thông qua hộp chọn (`Combobox`):

- **Beginner:** Bàn cờ kích thước 5×5, có 4 quả mìn.
- **Intermediate:** Bàn cờ kích thước 9×9, có 10 quả mìn.
- **Expert:** Bàn cờ kích thước 16×16, có 40 quả mìn.

2.2.2 Thao tác tương tác

Người chơi có thể tương tác với các ô trên bàn cờ bằng những thao tác sau:

- **Chuột trái** (`Button-1`)**:** Mở ô (*reveal*).
- **Chuột phải** (`Button-3` / `Ctrl+Button-1`)**:** Cắm hoặc tháo cờ (`toggle_flag`).
- **Chế độ cắm cờ (Flag mode):** Có nút gạt để bật/tắt chế độ cắm cờ, giúp thao tác thuận tiện trên màn hình cảm ứng hoặc với chuột một nút.

2.2.3 Tính năng hỗ trợ (Reasoning Assistant)

- **Get a hint:** Tính toán và làm nổi bật ô được đề xuất tối ưu bằng viền vàng dày, đồng thời hiển thị lời giải thích suy luận chi tiết trong bảng bên phải.
- **Play highlighted move:** Tự động thực hiện nước đi đang được gợi ý, gồm mở ô an toàn hoặc cắm cờ.
- **Play guess (may lose):** Đổi nhãn nút khi nước đi mang tính suy đoán và cảnh báo rủi ro trước khi người chơi xác nhận mạo hiểm.

2.3 Phân tách dữ liệu

Kiến trúc hệ thống áp dụng các nguyên tắc sau để bảo đảm mô-đun gợi ý chỉ sử dụng thông tin mà người chơi và AI được phép quan sát:

- **Phân tách thông tin:** Kiến trúc hệ thống tách biệt trạng thái thực tế của bàn cờ do Engine quản lý với thông tin được phép quan sát bởi AI và giao diện người dùng (GUI).
- **Giới hạn quyền truy cập:** Mô-đun gợi ý (`hints.py`) không truy cập trực tiếp vào đối tượng `Board` hoặc tập hợp `_mines`, nhằm tránh sử dụng thông tin về vị trí mìn chưa được tiết lộ.
- **Cung cấp dữ liệu an toàn:** Engine tạo một đối tượng `Snapshot` bất biến để cung cấp dữ liệu cho mô-đun gợi ý. Trong đối tượng này, các ô chưa mở được biểu diễn bằng `None`, giúp ẩn thông tin về vị trí mìn.
- **Đảm bảo tính khách quan của thuật toán:** AI đưa ra gợi ý dựa trên trạng thái quan sát được của bàn cờ và các quy tắc suy luận logic, thay vì truy cập trực tiếp vào dữ liệu mìn ẩn.

3 THUẬT TOÁN MỞ RỘNG VÙNG AN TOÀN

3.1 Lựa chọn thuật toán: Breadth-First Search (BFS) vs Depth-First Search (DFS)

Trong bài toán mở rộng vùng an toàn (Flood-fill) của Minesweeper, mục tiêu là tìm và lật mở toàn bộ thành phần liên thông gồm các ô có số đếm mìn bằng 0 (`clue = 0`) xuất phát từ ô được nhấp, cùng với đường biên gồm các ô mang số (`clue > 0`) bao quanh vùng đó.

Về mặt lý thuyết đồ thị, cả **Tìm kiếm theo chiều rộng (BFS)** và **Tìm kiếm theo chiều sâu (DFS)** đều có thể duyệt toàn bộ vùng cần mở với độ phức tạp thời gian $\mathcal{O}(N)$. Tuy nhiên, khi xét đến cách cài đặt trong Python, cấu trúc dữ liệu và khả năng trực quan hóa thuật toán, hai phương pháp có những điểm khác biệt:

3.1.1 Bảng so sánh đặc tính kỹ thuật: BFS và DFS trong Flood-fill

Bảng 3.1: *So sánh đặc tính kỹ thuật giữa BFS và DFS.*

Tiêu chí so sánh	Breadth-First Search (BFS)	Depth-First Search (DFS)
Cấu trúc dữ liệu chính	Hàng đợi FIFO (`Queue` hoặc `collections.deque`).	Ngăn xếp LIFO (`Stack` hoặc ngăn xếp lời gọi hàm).
Nguy cơ lỗi thời gian chạy	Không bị giới hạn bởi độ sâu đệ quy vì có thể duyệt lặp với hàng đợi trên heap; bộ nhớ hàng đợi vẫn tăng theo số ô cần xử lý.	DFS đệ quy có nguy cơ gây `RecursionError`. Python mặc định giới hạn độ sâu đệ quy, vùng số 0 lớn có thể vượt giới hạn này. DFS lặp với ngăn xếp tường minh tránh được giới hạn đệ quy.
Thứ tự lan tỏa	Lan tỏa theo từng lớp khoảng cách. Các ô gần ô bắt đầu được xử lý trước, tạo cảm giác như các vòng sóng mở rộng ra ngoài.	Đi sâu theo một nhánh trước rồi quay lui; hình thái lan tỏa phụ thuộc vào thứ tự duyệt các ô lân cận.
Khả năng minh họa từng bước	Trực quan và dễ theo dõi: hàng đợi FIFO cho thấy rõ các ô đang chờ xử lý, thuận tiện lập bảng mô phỏng (*dry run*).	Khó minh họa hơn vì quá trình đẩy và lấy ô khỏi ngăn xếp liên tục đổi nhánh, đặc biệt với DFS đệ quy.
Độ phức tạp thời gian và không gian	Thời gian $\mathcal{O}(N)$; không gian phụ trợ tối đa $\mathcal{O}(N)$.	Thời gian $\mathcal{O}(N)$; không gian phụ trợ tối đa $\mathcal{O}(N)$, phụ thuộc vào độ sâu tìm kiếm hoặc kích thước ngăn xếp.

3.1.2 Lý do lựa chọn BFS

1. **An toàn và ổn định khi cài đặt:** BFS dạng lặp không phụ thuộc vào giới hạn đệ quy của trình thông dịch Python. DFS cũng có thể được cài đặt lặp bằng ngăn xếp tường minh (`stack = []`), nhưng hàng đợi lặp là cách biểu diễn trực tiếp, dễ hiểu cho bài toán Flood-fill.
2. **Cơ chế lan tỏa tự nhiên, thân thiện với trải nghiệm người dùng (UX):** Hiệu ứng mở ô trong Minesweeper thường tạo cảm giác lan tỏa từ vị trí nhấp chuột. Với nguyên lý FIFO, BFS xử lý hết các ô ở khoảng cách d trước khi mở rộng sang các ô ở khoảng cách $d+1$, nhờ đó mô phỏng rõ sự lan tỏa theo từng lớp.
3. **Hiệu năng với `collections.deque`:** Phép lấy phần tử đầu hàng đợi bằng `popleft()` có độ phức tạp $\mathcal{O}(1)$; trong khi đó, dùng `list.pop(0)` cần dịch chuyển các phần tử còn lại và có thể tốn $\mathcal{O}(N)$ cho mỗi lần lấy.
4. **Phục vụ mục tiêu giáo dục và giải thích thuật toán:** BFS tạo thứ tự duyệt theo lớp rõ ràng, giúp nhóm dễ xây dựng bảng mô phỏng từng bước (*dry run*) ở mục 3.3 và đáp ứng yêu cầu *"document your algorithm step by step"* của bài tập lớn.

3.2 Mã giả cho BFS

Thuật toán mở rộng vùng an toàn được cài đặt trong phương thức `Board.reveal()` thuộc mô-đun `engine.py`. Đây là giải thuật tìm kiếm theo chiều rộng dạng lặp, sử dụng hàng đợi hai đầu `collections.deque` để tự động mở toàn bộ vùng các ô liên thông có số đếm mìn bằng 0 và dừng tại các ô biên mang số.

```
Input  : start_cell = (r, c)
Output  Tuple các ô đã thay đổi trạng thái
:
if status là won hoặc lost, hoặc start_cell thuộc (flags hợp revealed) then
    return () ;
end
if status = ready then
    gọi _place_mines(start_cell) ;                    // Bảo vệ ô đầu tiên và các ô lân cận
    status ← playing ;
end
if start_cell thuộc _mines then
    status ← lost ;
    exploded ← start_cell ;
    return (start_cell,) ;
end
Khởi tạo queue ← deque([start_cell]), queued ← {start_cell}, changed ← [] ;
while queue không rỗng do
    current ← queue.popleft() ;
    if current thuộc (flags hợp revealed) then
        continue ;
    end
    thêm current vào revealed và changed ;
    if _counts[current] = 0 then
        foreach neighbor trong neighbors(rows, cols, current) do
            if neighbor không thuộc (queued hợp flags hợp revealed) then
                thêm neighbor vào queued ;
                queue.append(neighbor) ;
            end
        end
    end
end
if số phần tử của revealed bằng rows nhân cols trừ mine_count then
    status ← won ;
end
return tuple(changed) ;
```

3.3 Mô phỏng thuật toán BFS

Xét bàn cờ 3×3 có một quả mìn tại góc trên bên trái, tọa độ $(1,1)$. Trong ví dụ này, tọa độ được ghi theo hệ 1-indexed để thuận tiện quan sát. Người chơi mở ô ở góc đối diện, $(3,3)$; ô này có số đếm mìn lân cận bằng 0. Các ô lân cận được xét theo thứ tự hàng trước, cột sau (row-major order). Bảng dưới đây mô tả trạng thái hàng đợi sau từng bước lặp:

Bảng 3.2: *Trạng thái hàng đợi sau từng bước lặp BFS.*

Bước	Ô lấy ra / clue	Các ô được thêm vào hàng đợi	Trạng thái hàng đợi sau bước
1	`(3,3) / 0`	`(2,2), (2,3), (3,2)`	`[(2,2), (2,3), (3,2)]`
2	`(2,2) / 1`	Không mở rộng vì clue > 0 (ô biên).	`[(2,3), (3,2)]`
3	`(2,3) / 0`	`(1,2), (1,3)`	`[(3,2), (1,2), (1,3)]`
4	`(3,2) / 0`	`(2,1), (3,1)`	`[(1,2), (1,3), (2,1), (3,1)]`
5	`(1,2) / 1`	Không mở rộng vì clue > 0 (ô biên).	`[(1,3), (2,1), (3,1)]`
6	`(1,3) / 0`	Không có; các ô lân cận đã được đưa vào hàng đợi.	`[(2,1), (3,1)]`
7	`(2,1) / 1`	Không mở rộng vì clue > 0 (ô biên).	`[(3,1)]`
8	`(3,1) / 0`	Không có; các ô lân cận đã được đưa vào hàng đợi.	Hàng đợi rỗng; thuật toán kết thúc.

Kết quả thực thi: Cả 8 ô an toàn trên bàn cờ được lật mở thành công, nên trò chơi đạt điều kiện chiến thắng.
Nhận xét: Quả mìn tại $(1,1)$ không bao giờ được đưa vào hàng đợi. Các ô kề với nó là những ô biên có clue bằng 1, chẳng hạn $(1,2)$, $(2,2)$ và $(2,1)$. Theo thuật toán, các ô có clue lớn hơn 0 không mở rộng sang các ô lân cận.

3.4 Đánh giá độ phức tạp thuật toán

Gọi $N = R \times C$ là tổng số ô trên bàn cờ.
Độ phức tạp thời gian: $\mathcal{O}(N)$. Nhờ tập hợp băm `queued`, mỗi ô được đưa vào hàng đợi tối đa một lần. Với mỗi ô lấy ra, thuật toán duyệt tối đa 8 ô lân cận theo lân cận Moore; do đó tổng số lượt xét lân cận không vượt quá $8N$. Các thao tác `popleft()` và `append()` trên `collections.deque` có độ phức tạp trung bình $\mathcal{O}(1)$, nên tổng thời gian trong trường hợp xấu nhất là tuyến tính theo số ô.

Độ phức tạp không gian phụ trợ: $\mathcal{O}(N)$. Bộ nhớ phụ trợ chủ yếu được dùng cho hàng đợi `queue` và tập `queued`. Trong trường hợp xấu nhất, mỗi cấu trúc có thể lưu tối đa N tọa độ. Bộ nhớ này nằm trên heap, không làm tăng độ sâu call stack như cách cài đặt DFS đệ quy.

4 PHÂN TÍCH HÀM HEURISTIC VÀ SUY LUẬN LOGIC (LOGICAL HINT RULES & HEURISTICS)

4.1 Cơ chế suy diễn logic (Deduction Rules)

Hệ thống xây dựng các ràng buộc dạng phương trình nguyên. Với mỗi ô đã mở có số đếm c, gọi $U(c)$ là tập các ô lân cận còn đang ẩn. Mỗi ô ẩn u được gán biến nhị phân $x(u)$, trong đó $x(u) = 1$ nếu ô đó có mìn và $x(u) = 0$ nếu ô đó an toàn:

$$\sum_{u \in U(c)} x(u) = c, \qquad x(u) \in \{0,1\}.$$

Trong quá trình suy diễn, các ô đã được xác định chắc chắn an toàn ($x = 0$) hoặc chắc chắn có mìn ($x = 1$) được rút gọn khỏi ràng buộc. Sau bước chuẩn hóa, U là tập ô ẩn chưa xác định còn lại và k là số mìn còn lại trong tập đó.

- **Rule 1 – Số mìn còn lại bằng 0 ($k = 0$):** Tất cả các ô còn lại trong U chắc chắn an toàn, tức là $x(u) = 0$ với mọi $u \in U$. Hệ thống có thể gợi ý mở các ô này.
- **Rule 2 – Số ô còn lại bằng số mìn ($k = |U|$):** Tất cả các ô còn lại trong U chắc chắn có mìn, tức là $x(u) = 1$ với mọi $u \in U$. Hệ thống có thể gợi ý cắm cờ các ô này.
- **Rule 3 – Sai phân tập con (Subset difference constraints):** Nếu tập ô ẩn của ràng buộc A là tập con thực sự của tập ô ẩn của ràng buộc B ($U_A \subset U_B$), xét tập hiệu $D = U_B \setminus U_A$. Số mìn trong D là $\Delta k = k_B - k_A$. Nếu $\Delta k = 0$, toàn bộ ô trong D an toàn; nếu $\Delta k = |D|$, toàn bộ ô trong D có mìn.

4.2 Hàm Heuristic ước lượng rủi ro cục bộ (Estimated Local Risk)

Khi các luật suy diễn logic không tìm thêm được ô nào chắc chắn an toàn hoặc chắc chắn có mìn, hệ thống dùng heuristic để xếp hạng các nước đi theo mức rủi ro ước lượng.

Với một ô ứng viên u tiếp giáp m ràng buộc $C_1, C_2, \ldots, C_m$, gọi U_i là tập ô ẩn trong ràng buộc thứ i và k_i là số mìn còn lại trong tập đó. Mật độ mìn dư của ràng buộc thứ i được tính như sau:

$$d_i = \frac{k_i}{|U_i|}.$$

Điểm rủi ro cục bộ của ô u là trung bình cộng các mật độ này:

$$\text{Risk}(u) = \frac{1}{m} \sum_{i=1}^{m} d_i.$$

Nếu ô u nằm biệt lập, không giáp ô số nào, hệ thống ước lượng rủi ro theo mật độ toàn cục. Gọi M là tổng số mìn, $\text{Mines}_{\text{known}}$ là tập các ô đã được xác định chắc chắn có mìn và Hidden là tập các ô chưa mở:

$$\text{Risk}_{\text{global}} = \frac{M - |\text{Mines}_{\text{known}}|}{|\text{Hidden}| - |\text{Mines}_{\text{known}}|}.$$

Hệ thống ưu tiên chọn ô có $\text{Risk}(u)$ nhỏ nhất. Khi nhiều ô có cùng điểm rủi ro, thứ tự phá vỡ hòa như sau:

1. Ưu tiên ô có điểm rủi ro thấp nhất.
2. Nếu rủi ro bằng nhau, ưu tiên ô có nhiều ô lân cận còn ẩn nhất. Đây là đại diện gần đúng cho *Information Gain*, với kỳ vọng ô an toàn sẽ mở ra thêm nhiều thông tin.
3. Nếu vẫn hòa, chọn theo tọa độ hàng rồi cột tăng dần để kết quả có tính tất định.

4.3 Đánh giá tính hợp lệ của Heuristic

Độ phức tạp tính toán (Hint Cost): Gọi $N = R \times C$ là tổng số ô. Ban đầu có tối đa $\mathcal{O}(N)$ ràng buộc. Nếu mỗi vòng suy diễn tìm được ít nhất một ô mới, số vòng lặp tối đa là $\mathcal{O}(N)$. Với tối đa $\mathcal{O}(N)$ ràng buộc đang được giữ ở mỗi vòng, có $\mathcal{O}(N^2)$ cặp ràng buộc cần so sánh; kiểm tra quan hệ tập con có thể tốn đến $\mathcal{O}(N)$ trong trường hợp xấu nhất. Vì vậy, chi phí mỗi vòng có thể bị chặn trên bởi $\mathcal{O}(N^3)$, tương ứng cận trên bảo thủ $\mathcal{O}(N^4)$ cho toàn bộ quá trình. Đây là phân tích độ phức tạp lý thuyết; cận trên này không phải số liệu benchmark. Thời gian chạy thực tế cần được đo riêng và phụ thuộc vào cách cài đặt cũng như môi trường chạy.

Giới hạn của bộ luật suy diễn: Các luật trên không tạo thành một bộ giải CSP tổng quát: hệ thống không giải toàn bộ hệ phương trình bằng khử Gauss–Jordan hoặc tìm kiếm nhánh-cận/SAT. Vì vậy, trong một số tình huống giao thoa phức tạp, hệ thống không thể chứng minh chắc chắn trạng thái của các ô và phải chuyển sang chế độ suy đoán (`Guess mode`).

Giới hạn của hàm heuristic: Điểm rủi ro trung bình cục bộ chỉ là một giá trị xấp xỉ dùng để xếp hạng (*ranking score*); nó không mô tả chính xác phân phối xác suất hậu nghiệm chung (*joint posterior probability*), vì các tập ô lân cận trong những ràng buộc khác nhau có thể phụ thuộc lẫn nhau.

5 Kết quả thực nghiệm và kiểm thử hệ thống

5.1 Phương pháp kiểm thử tự động (Automated Checks)

Dự án tích hợp bộ kiểm thử gồm 32 test case, được thực thi bằng `unittest`. Các kiểm thử tập trung vào tính đúng đắn của BFS, độ an toàn của các suy luận heuristic và cơ chế bảo vệ lượt đi đầu tiên.

- **Đánh giá tính đúng đắn của BFS (BFS region oracle):** Kiểm tra vét cạn toàn bộ 36 cấu hình có 2 quả mìn trên bàn cờ 3×3. Với mỗi cấu hình, thử mở tất cả 7 ô an toàn ban đầu, tổng cộng 252 lần mở. Kết quả: 100% vùng mở rộng của BFS khớp với mô hình tính toán điểm bất động độc lập (*region oracle*).
- **Tính không mâu thuẫn của heuristic (Deduction soundness):** Kiểm thử 36 cấu hình với 128 tập con trạng thái mở mỗi cấu hình, tổng cộng $36 \times 128 = 4{,}608$ trạng thái. Kết quả: không có suy luận an toàn nào chọn nhầm ô mìn và không có suy luận mìn nào gán nhầm cho ô an toàn.
- **Bảo vệ lượt đi đầu tiên (First-move protection):** Kiểm thử với 15 seed ngẫu nhiên trên cả ba kích thước bàn cờ. Kết quả: 100% ô được nhấp đầu tiên và các ô lân cận đều không có mìn.

Các kết quả kiểm thử, bao gồm các tỷ lệ 100%, chỉ áp dụng trong phạm vi cấu hình đã nêu; chúng không chứng minh thuật toán đúng với mọi bàn cờ có thể có.

5.1.1 Cách thực thi kiểm thử

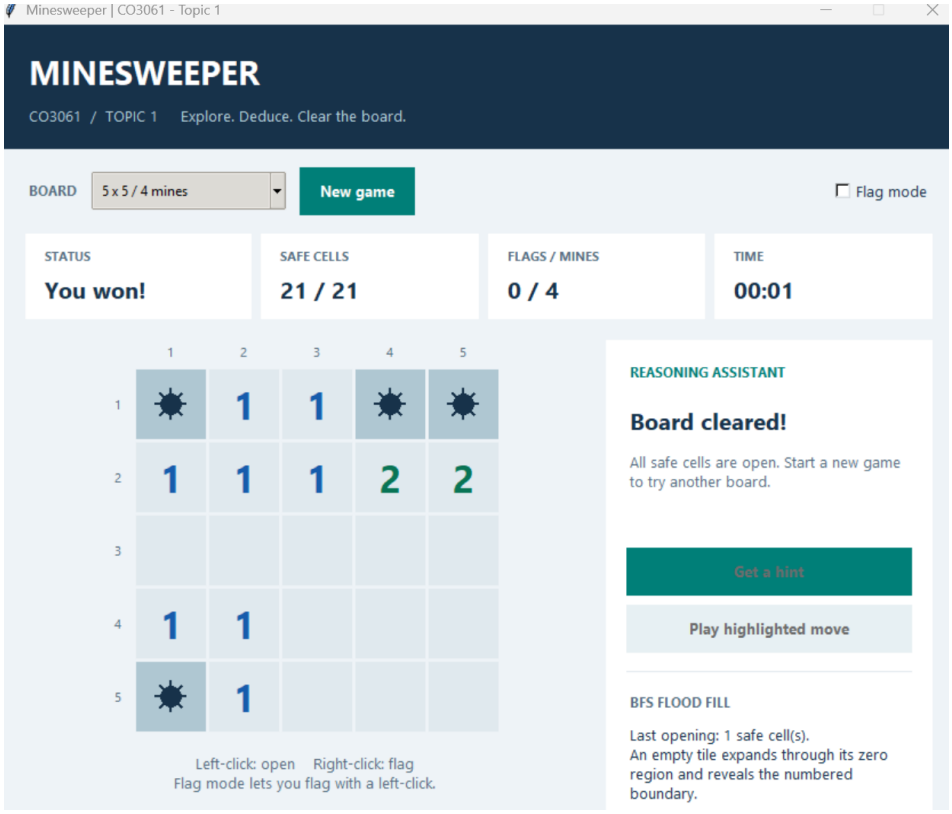
Chạy các lệnh sau từ thư mục gốc của dự án:

```
python -m unittest discover -s tests -v
python gui_smoke.py
```

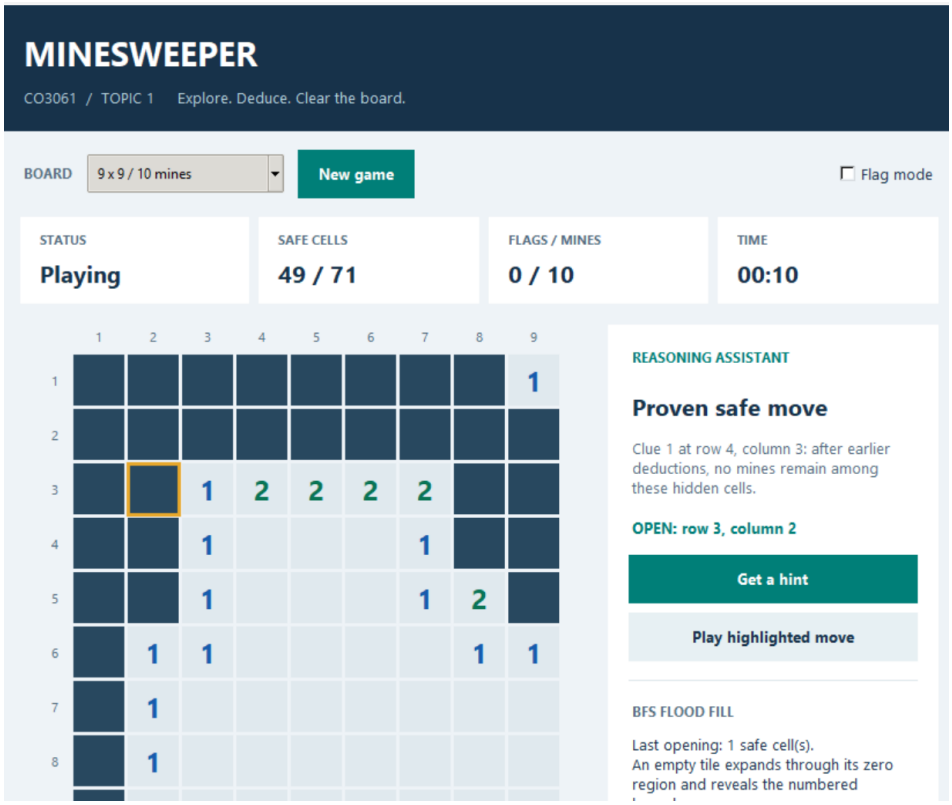
5.2 Hình ảnh minh họa giao diện thực tế

Các ảnh chụp màn hình dưới đây minh họa giao diện Minesweeper ở các trạng thái và kích thước bàn cờ khác nhau.
Nhận xét trực quan:

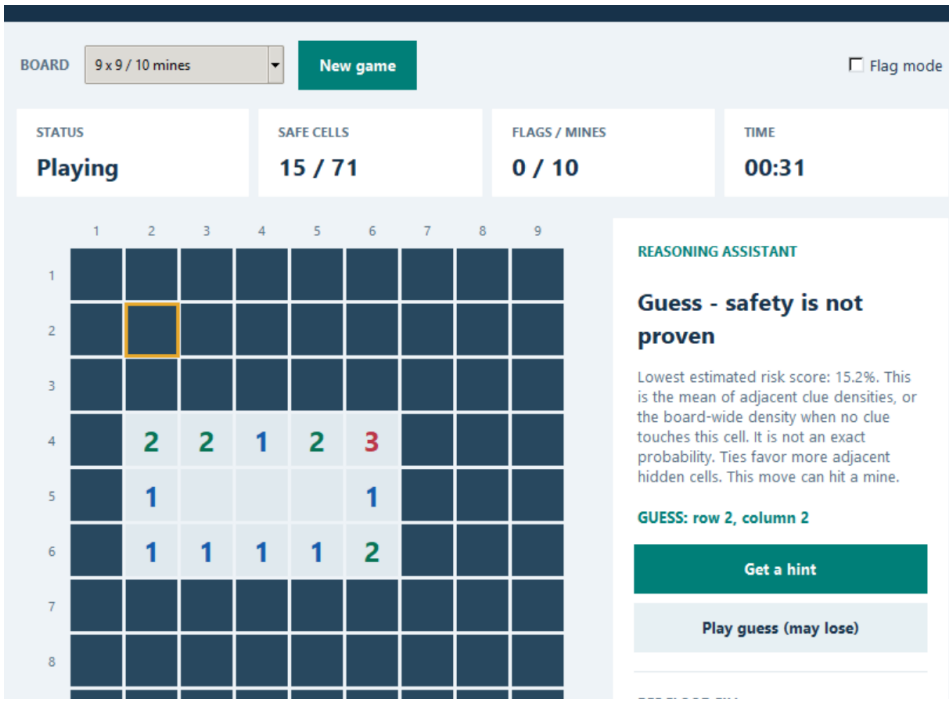
- Các số đếm từ 1 đến 8 có màu sắc tương phản trên nền ô cờ sáng, giúp dễ quan sát ở cả bàn cờ chiến thắng và bàn cờ đang chơi (Hình 5.1 và Hình 5.4).
- Kích thước ô cờ được điều chỉnh theo chế độ, từ 64 px ở bàn cờ 5×5 xuống 28 px ở bàn cờ 16×16, giúp bàn cờ nằm gọn trong vùng hiển thị (Hình 5.1 và Hình 5.4).
- Bảng Reasoning Assistant phân biệt nước đi an toàn đã được chứng minh (`Play highlighted move`) với nước đi suy đoán có rủi ro (`Play guess (may lose)`), tương ứng ở Hình 5.2 và Hình 5.3.
- Dù bàn cờ có kích thước 16×16, các ô đã mở và lá cờ vẫn được thể hiện trong vùng giao diện, giúp người chơi theo dõi trạng thái ván đấu (Hình 5.4).



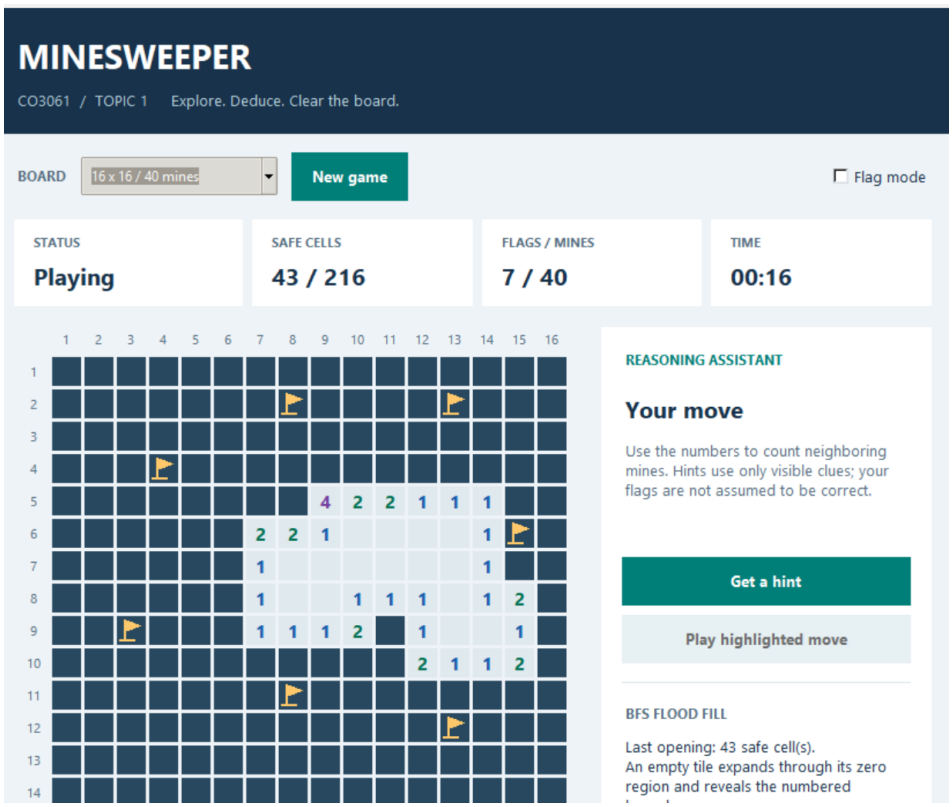
Hình 5.1: *Bàn cờ* 5×5 *sau khi hoàn thành ván đấu (*`WON`*).*



Hình 5.2: *Bàn cờ* 9×9 *với gợi ý nước đi an toàn đã được chứng minh (*`Proven safe move`*).*



Hình 5.3: *Chế độ suy đoán mạo hiểm (`Guess mode`) với cảnh báo rủi ro.*



Hình 5.4: *Bàn cờ* 16×16 *trong trạng thái đang chơi (`Playing`), hiển thị vùng đã mở và các lá cờ.*

5.3 Đánh giá độ ổn định và hướng dẫn thực thi mã nguồn

Độ ổn định: Trò chơi không sử dụng thư viện bên thứ ba, hoạt động độc lập trên các nền tảng hỗ trợ Python 3.10 trở lên, gồm Windows, Linux và macOS.

5.3.1 Hướng dẫn chạy mã nguồn

- **Trên Windows:** Nhấp đúp vào tệp `run.bat` hoặc mở terminal tại thư mục dự án và chạy:

```
python main.py
```

- **Chạy với seed cố định:** Ví dụ, để tái lập bàn cờ cấp độ 9 với seed 42, dùng lệnh:

```
python main.py --level 9 --seed 42
```

- **Biên dịch báo cáo PDF:** Tại thư mục gốc của dự án, chạy:

```
python build_report.py
```

6 TÀI LIỆU THAM KHẢO

[1] Đề bài Bài tập lớn: HCMUT – VNU HCM, CO3061: *Introduction to Artificial Intelligence – Homework 1: Topic 1 – Minesweeper*, Học kỳ 1, 2026.

[2] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Prentice Hall. (Chương về Constraint Satisfaction Problems và Search Algorithms).

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). The MIT Press. (Chương 20, mục 20.2 về Breadth-First Search).

[4] Kaye, R. (2000). Minesweeper is NP-complete. *The Mathematical Intelligencer, 22*(2), tr. 9–15.

[5] Studholme, C. (2000). *Minesweeper as a Constraint Satisfaction Problem*. Department of Computer Science, University of Toronto.

[6] Becerra, D. J. (2015). *Algorithmic Approaches to Playing Minesweeper*. [Bachelor's thesis, Harvard College].

[7] Python Software Foundation. *Tkinter – Python interface to Tcl/Tk*, thư viện chuẩn Python. URL: `https://docs.python.org/3/library/tkinter.html`.

[8] Python Software Foundation. *collections.deque – High-performance container datatypes*, thư viện chuẩn Python. URL: `https://docs.python.org/3/library/collections.html#collections.deque`.

[9] Python Software Foundation. *unittest – Unit testing framework*, thư viện chuẩn Python. URL: `https://docs.python.org/3/library/unittest.html`.